

- $\delta: Q \times \Sigma \longrightarrow Q$ is the transition function

e.g., $\delta(q_0, 0) = q_1$

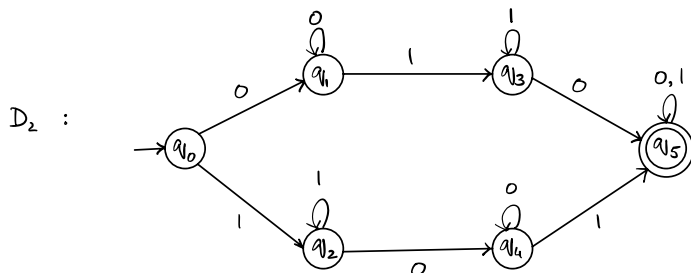
- q_0 is the starting state

- F is the set of accepting states, $F \subseteq Q$

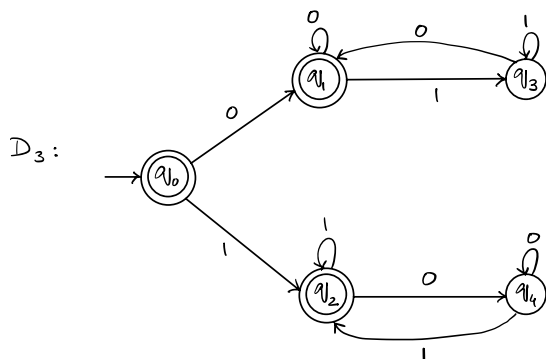
δ	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_0

e.g.,

$$L_2 = \{w \in \Sigma^* \mid w \text{ contains } 01 \text{ and } 10\}$$



$$L_3 = \{w \in \Sigma^* \mid w \text{ has equal occurrences of } 01 \text{ and } 10\}$$



Let us formalize what it means for a DFA D to accept a string w .

Define the notion of an **extended transition function** $\hat{\delta}$ that works on arbitrary strings in Σ^* rather than symbols $\sigma \in \Sigma$.

$\hat{\delta}: Q \times \Sigma^* \longrightarrow Q$ So, e.g., $\hat{\delta}(q_0, 001) = q_3$ for above DFA D_2 . Why?

result is the state q_1 we reach after processing 00 .

$$\hat{\delta}(q_0, 001) = \delta(\underbrace{\hat{\delta}(q_0, 00)}_{q_1}, 1) = \delta(\delta(\hat{\delta}(q_0, 0), 0), 1) = \delta(\delta(\delta(q_0, 0), 0), 1)$$

We broke 001 into $00 \& 1$. The extended function processes the initial string 00 recursively, while the final 1 is processed by δ .

$$\delta(\delta(\delta(q_0, 0), 0), 1) = \delta(\delta(q_1, 0), 1) = \delta(q_1, 1) = q_3 \rightarrow \text{Since } q_3 \text{ is not an accepting state, } 001 \text{ is rejected.}$$

Similarly, 1001 is accepted since $\hat{\delta}(q_0, 1001) = q_2 \in F$.

$$\begin{aligned}\hat{\delta}(q_0, 1001) &= \delta(\hat{\delta}(q_0, 100), 1) = \delta(\delta(\hat{\delta}(q_0, 10), 0), 1) = \delta(\delta(\delta(\hat{\delta}(q_0, 1), 0), 0), 1) \\ &= \delta(\delta(\delta(\delta(q_0, 1), 0), 0), 1) = \delta(\delta(\delta(q_2, 0), 0), 1) = \delta(\delta(q_4, 0), 1) = \delta(q_4, 1) = q_2.\end{aligned}$$

So, in general, for a string $w \in \Sigma^*$, with $w = w_1 w_2 \dots w_k$, where $w_i \in \Sigma$.

$$\begin{aligned}\hat{\delta}(q_0, w = w_1 w_2 \dots w_k) &= \delta(\hat{\delta}(q_0, w_1 w_2 \dots w_{k-1}), w_k) \\ &= \dots \\ &= \delta(\delta(\dots \delta(q_0, w_1), \dots), w_{k-1}), w_k)\end{aligned}$$

Finally, a DFA accepts a string w if $\hat{\delta}(q_0, w) = q_i \in F$

For a DFA D , the language of D , $L(D)$ is the set of strings accepted by D .

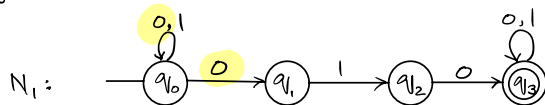
A language is regular if and only if there exists a DFA that accepts it.

Non-Deterministic Finite Automata (NFA)

We try to generalize the DFA model, by allowing the following:

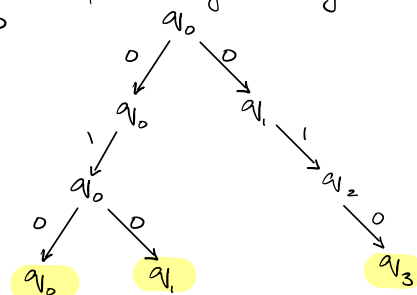
1. Allow $\delta(q, \sigma)$, $\sigma \in \Sigma$, to specify zero or more than one state
2. Allow λ -transitions

e.g. $L = \{w \in \Sigma^* \mid w \text{ contains } 010\}$

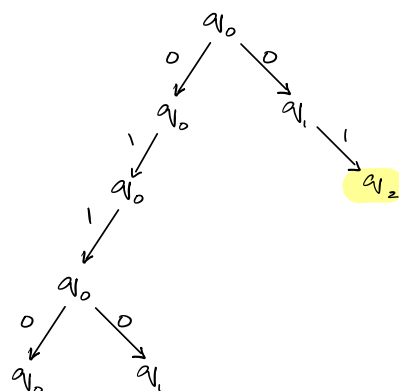


q_0 has non-determinism, multiple transitions on 0.
 q_1 has no transition on 0
 q_2 " " " " " 1

Take all possible paths for a given string w .
 Accept if there exists atleast one path that ends in an accepting state.
 We can keep track of all paths by building a computation history tree.
 e.g., on string 010



Starting from q_0 there exist 3 paths that completely process 010, one of which ends in $q_3 \in F$. Hence, string is accepted.



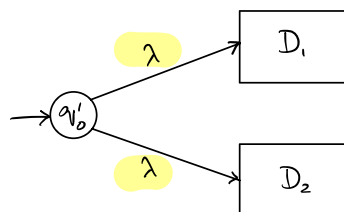
$w = 0110$ is not accepted.

Note path just dies off after only processing 01, since no transition on 1 from q_1 .

We have argued in class regular languages are closed under complement operation. What about union?

i.e., given two DFAs D_1 & D_2 can we construct a DFA that accepts $L(D_1) \cup L(D_2)$?

Here is an NFA construction:



Add a new state q'_0 with λ -transitions to starting states of D_1 & D_2 .

But this is not a DFA!
What about intersection?

Def An NFA N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

$\delta: Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \mathcal{P}(Q)$
Powerset of Q .